

ASTRA FLIGHT CONTROLLER

Complete Technical Reference Manual

STM32F411CEU6 • ICM-42688-P • USB Type-C • 100 mm × 100 mm PCB

Table of Contents

- Chapter 1 – Introduction
- Chapter 2 – System Overview
- Chapter 3 – Power System
- Chapter 4 – STM32F411CEU6 Microcontroller
- Chapter 5 – ICM-42688-P Inertial Measurement Unit (IMU)
- Chapter 6 – USB Type-C Interface
- Chapter 7 – External Interfaces and Connectors
- Chapter 8 – Indicators and User Interface
- Chapter 9 – PCB Layout & Design Considerations
- Chapter 10 – Power Supply Design
- Chapter 11 – Firmware & Programming

Chapter 1 – Introduction

1.1 Project Overview

1.1.1 Introduction

The Astra Flight Controller is a custom-designed flight control board developed for multirotor unmanned aerial vehicles (UAVs). It is built around the STM32F411CEU6 microcontroller and the ICM-42688-P six-axis Inertial Measurement Unit (IMU), providing a compact, high-performance platform capable of running modern flight control firmware such as Betaflight, INAV, or custom-developed firmware.

Unlike commercial flight controllers, Astra has been designed from the ground up with a strong focus on learning, customization, and hardware understanding. Every component on the PCB has been selected manually, every connection has been verified, and the entire schematic and PCB layout have been created using KiCad.

This document serves as the complete technical reference for the Astra Flight Controller. It explains the design choices, hardware architecture, power distribution, component selection, PCB layout practices, assembly recommendations, and debugging procedures.

1.1.2 Main Features

The Astra Flight Controller includes the following hardware features:

- STM32F411CEU6 ARM Cortex-M4 Microcontroller
- ICM-42688-P 6-axis IMU
- MP2359DJ High-Efficiency 5 V Buck Converter
- AP2112K 3.3 V Low-Dropout Voltage Regulator
- USB Type-C Interface
- SWD Programming Header
- XT30 Battery Input
- Active Buzzer Driver Circuit
- Status LED
- External UART Expansion
- Dedicated SPI Interface for the IMU
- Compact PCB designed within a 100 mm × 100 mm manufacturing limit

1.1.3 Applications

The Astra Flight Controller is suitable for:

- FPV Racing Drones
- Freestyle Quadcopters
- Educational Robotics
- Autonomous UAV Development
- Research Projects
- Flight Control Algorithm Development
- Embedded Systems Learning

1.2 Design Objectives

The purpose of the Astra Flight Controller project extends beyond simply building another flight controller. The project was undertaken to gain practical experience in professional PCB design and embedded hardware development while producing a fully functional and manufacturable controller.

The primary design objectives were:

Educational Value

To understand every subsystem involved in designing a modern flight controller, including power regulation, sensor integration, high-speed digital communication, PCB layout, and firmware programming.

Reliability

To create a stable hardware platform using industry-proven components capable of operating under vibration, electrical noise, and varying power conditions encountered in UAV applications.

Manufacturability

To ensure the PCB can be fabricated using standard manufacturing services such as JLCPCB without requiring advanced fabrication processes or expensive custom options.

Serviceability

To make debugging and firmware development straightforward by including:

- Standard SWD programming interface
- Clearly labelled connectors
- Modular circuit blocks
- Easy-to-understand schematic organization

Expandability

The design intentionally leaves room for future improvements, including:

- Barometer support
- Magnetometer support
- Blackbox Flash Memory
- On-board OSD
- Bluetooth or Wi-Fi telemetry
- CAN bus peripherals
- Additional UART interfaces

1.2.1 Project Specifications

Parameter	Specification
Project Name	Astra Flight Controller
PCB Software	KiCad 10
PCB Size	$\leq 100 \text{ mm} \times 100 \text{ mm}$
MCU	STM32F411CEU6
IMU	ICM-42688-P
Main Input Voltage	Battery via XT30
Regulated Outputs	5 V, 3.3 V
Programming Interface	SWD

Parameter	Specification
USB Interface	USB Type-C
Intended Manufacturer	JLCPCB

Chapter Summary: *This chapter introduced the Astra Flight Controller project, described its overall purpose, listed its major hardware features, and defined the engineering goals that guided the design. The following chapters examine each subsystem in detail, beginning with the overall hardware architecture and system operation.*

Chapter 2 – System Overview

2.1 Hardware Architecture

2.1.1 Introduction

The Astra Flight Controller is designed around a modular hardware architecture, where each functional block performs a specific task while interacting with the others through clearly defined interfaces. This modular approach simplifies debugging, improves reliability, and allows future upgrades without requiring a complete redesign.

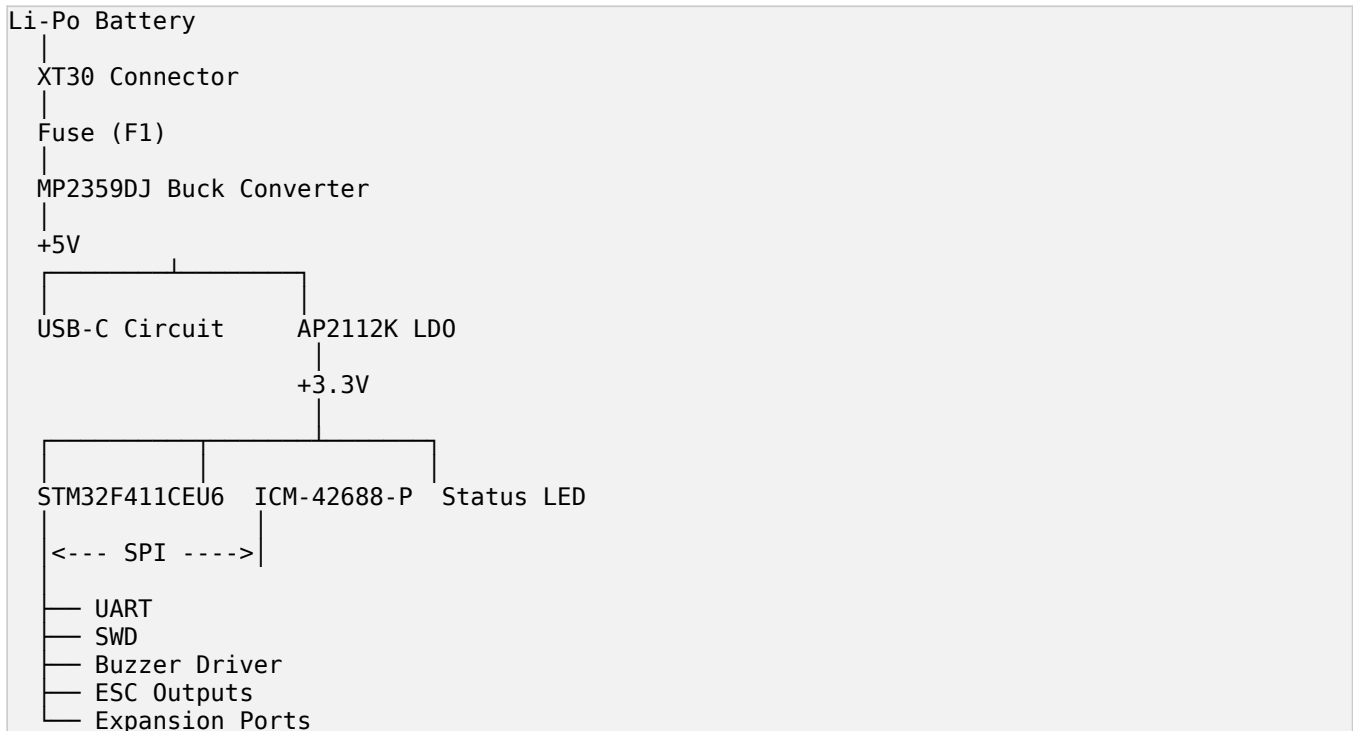
The hardware is divided into seven major subsystems:

- Power Management
- Processing Unit
- Sensor System
- Communication Interfaces
- User Interface
- Programming & Debugging
- External Connections

Each subsystem operates independently while contributing to the overall functionality of the flight controller.

2.1.2 Functional Block Diagram

The overall operation of the Astra Flight Controller can be represented as follows:



This block diagram shows the overall power flow and signal flow through the flight controller.

2.1.3 Hardware Modules

Power Management

Responsible for converting battery voltage into stable regulated voltages used by the board.

Major Components:

- XT30 Battery Connector
- Resettable Fuse
- MP2359DJ Buck Converter
- AP2112K 3.3V Regulator Filtering Capacitors

Outputs:

- +5V Rail
- +3.3V Rail

Processing Module

The processing module is the "brain" of the flight controller. Main Component: STM32F411CEU6.

Responsibilities:

- Reads sensor data
- Executes flight control algorithms
- Controls motors
- Handles USB communication
- Processes receiver input
- Generates PWM/DShot signals
- Controls LEDs and buzzer

Sensor Module

The sensor subsystem provides real-time motion data. Main Sensor: ICM-42688-P.

Measurements: 3-axis Accelerometer, 3-axis Gyroscope. Communication: SPI Interface.

The IMU continuously measures angular velocity and acceleration, enabling the firmware to estimate the drone's orientation and movement.

Communication Module

Communication interfaces allow the controller to interact with external devices. Interfaces include USB Type-C, SWD Programming, SPI, and UART. These interfaces are used for firmware flashing, debugging, telemetry, and configuration.

User Interface

The user interface consists of simple hardware that provides status information and manual interaction.

Components: Status LED, Push Button, Active Buzzer. These components assist during programming, calibration, troubleshooting, and normal operation.

External Interfaces

The controller provides several connectors for integration with other systems: Battery Input, ESC Outputs, UART Expansion, SWD Header, USB-C Connector. These connectors enable communication with motors, sensors, programming tools, and external peripherals.

2.2 System Operation

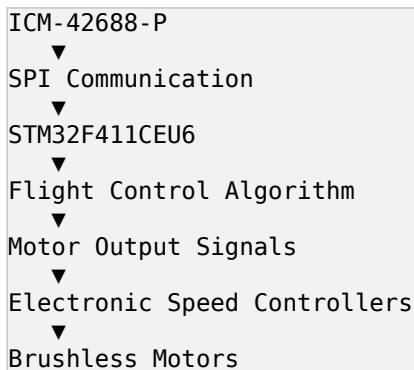
2.2.1 Power-Up Sequence

When battery power is applied, the following sequence occurs:

- Step 1: Battery voltage enters through the XT30 connector.
- Step 2: The resettable fuse protects the board from excessive current.
- Step 3: The MP2359DJ converts battery voltage into a regulated +5V supply.
- Step 4: The AP2112K converts +5V into +3.3V.
- Step 5: The STM32F411CEU6 receives stable power.
- Step 6: The crystal oscillator starts generating the system clock.
- Step 7: The microcontroller boots from Flash memory.
- Step 8: The SPI interface initializes the ICM-42688-P.
- Step 9: Sensors begin transmitting motion data.
- Step 10: The flight controller enters normal operating mode.

2.2.2 Data Flow

During flight, information continuously flows through the controller.



The entire control loop executes hundreds or even thousands of times per second, depending on the firmware configuration.

2.2.3 Clock Generation

The STM32F411CEU6 uses an external crystal oscillator for accurate timing. The external crystal provides stable CPU clock, accurate USB timing, reliable SPI communication, and consistent sensor sampling. Proper placement of the crystal and its load capacitors is essential for reliable operation.

2.2.4 Design Philosophy

Several engineering principles were followed throughout the design of the Astra Flight Controller:

Simplicity

The schematic is divided into functional blocks that are easy to understand and maintain.

Reliability

All power rails include local decoupling capacitors, short return paths, and proper grounding techniques.

Expandability

Unused GPIO pins and communication interfaces are available for future hardware additions.

Manufacturability

The PCB is designed to be compatible with standard manufacturing services such as JLCPCB while remaining within the 100 mm × 100 mm board size limit.

Chapter Summary: *This chapter introduced the overall architecture of the Astra Flight Controller, explaining how power, processing, sensing, communication, and external interfaces work together as a complete embedded system. It also described the power-up sequence and the flow of data from the IMU to the microcontroller and finally to the motor outputs. The next chapter examines the Power System in detail, beginning with the battery input, protection circuitry, and voltage regulation stages.*

Chapter 3 – Power System

3.1 Introduction

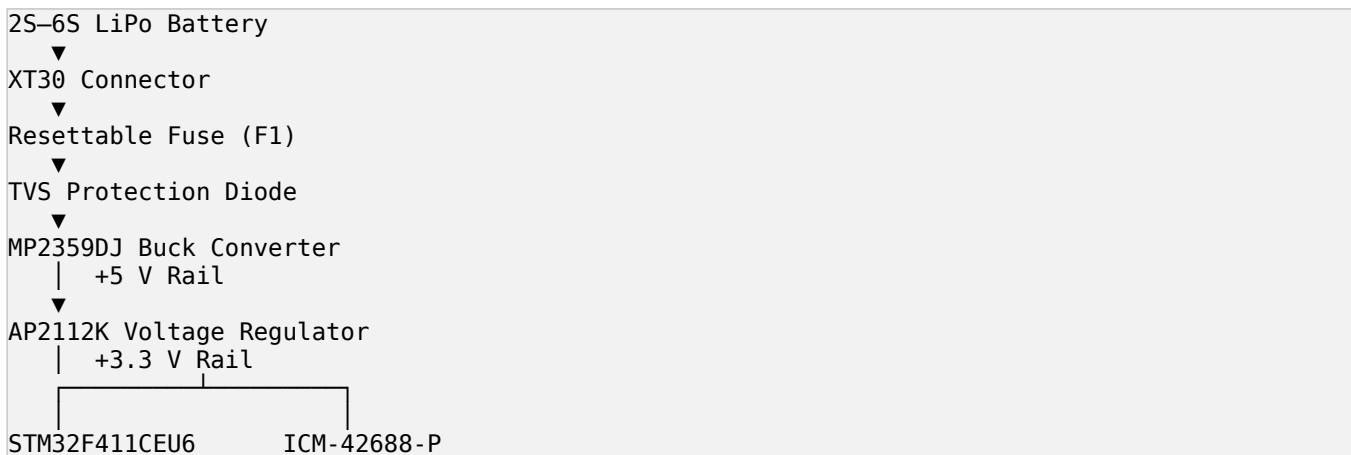
3.1.1 Overview

The power system is one of the most critical sections of the Astra Flight Controller. Every subsystem—including the microcontroller, IMU, USB interface, and peripherals—depends on a stable and clean power supply. A poorly designed power stage can lead to unstable sensor readings, processor resets, communication failures, or complete system failure during flight.

The Astra Flight Controller is designed to accept a 2S–6S LiPo battery as its primary power source. Since the battery voltage varies significantly depending on the number of cells and state of charge, the board incorporates dedicated power conversion and regulation stages to generate stable operating voltages.

3.1.2 Power Architecture

The complete power path of the flight controller is shown below.



The power is converted in two stages: Battery → 5 V using the MP2359DJ switching regulator, then 5 V → 3.3 V using the AP2112K low-dropout regulator. This architecture provides both high efficiency and a clean supply for sensitive digital electronics.

3.1.3 Supported Input Voltage

The controller supports the following battery configurations:

Battery	Nominal Voltage	Fully Charged
2S LiPo	7.4 V	8.4 V
3S LiPo	11.1 V	12.6 V
4S LiPo	14.8 V	16.8 V
5S LiPo	18.5 V	21.0 V
6S LiPo	22.2 V	25.2 V

The MP2359DJ is capable of regulating this wide input range into a stable 5 V output.

3.2 Power Rails

The Astra Flight Controller contains three main power domains.

VBAT

VBAT is the raw battery voltage entering the board through the XT30 connector. It is used as the input to the buck converter and is also monitored through a resistor divider connected to the STM32 ADC for battery voltage measurement.

+5 V Rail

The +5 V rail is generated by the MP2359DJ buck converter. This rail supplies the AP2112K voltage regulator, USB-related circuitry, the buzzer, and external peripherals requiring 5 V. Using a switching regulator keeps power loss low even when operating from a 6S battery.

+3.3 V Rail

The +3.3 V rail is generated by the AP2112K low-dropout regulator. It powers the STM32F411CEU6, the ICM-42688-P IMU, logic-level circuits, and digital communication interfaces. The AP2112K was selected because of its low output noise and excellent transient response, making it suitable for precision sensor applications.

3.2.1 Why Two Regulators?

Using a single linear regulator directly from the battery would result in excessive heat generation, especially at higher input voltages. Instead, the Astra Flight Controller uses a two-stage regulation approach:

- MP2359DJ Buck Converter — efficiently reduces battery voltage to 5 V.
- AP2112K LDO — produces a clean and stable 3.3 V output with minimal electrical noise.

This combination provides both efficiency and stable operation.

3.2.2 Power Design Considerations

Several engineering principles were followed during the design of the power system:

- High-current traces are kept short and wide.
- The buck converter is located close to the battery input.
- Input and output capacitors are placed near the MP2359DJ.
- The AP2112K is positioned close to the STM32 and IMU.
- All IC supply pins include local decoupling capacitors.
- The exposed pad of the STM32 is connected to the ground plane for improved thermal performance and electrical grounding.

Chapter Summary: *The Astra Flight Controller uses a robust two-stage power architecture to convert a 2S–6S LiPo battery into regulated 5 V and 3.3 V supplies. The MP2359DJ provides efficient primary voltage conversion, while the AP2112K supplies a clean 3.3 V rail for the microcontroller and IMU. This design balances efficiency, stability, and noise performance, providing a reliable foundation for all other subsystems.*

Chapter 4 – STM32F411CEU6 Microcontroller

4.1 Introduction

The STM32F411CEU6 is the primary processing unit of the Astra Flight Controller. It executes the flight control firmware, processes sensor data, generates motor outputs, and manages all communication interfaces.

Parameter	Specification
Manufacturer	STMicroelectronics
Device	STM32F411CEU6
CPU Core	ARM Cortex-M4F
Maximum Clock	100 MHz
Flash	512 KB
SRAM	128 KB
Voltage	1.7–3.6 V
Package	UFQFPN-48 (Exposed Pad)
Debug	SWD

Role

The STM32 reads IMU data through SPI, executes flight control algorithms, generates ESC outputs, monitors battery voltage, controls the LED and buzzer, manages USB communication, and supports firmware programming through SWD.

Power and Clock

The MCU operates from the regulated 3.3 V rail generated by the AP2112K regulator. All VDD pins connect to 3.3 V, all VSS pins and the exposed Pad 49 connect to the ground plane. An external crystal provides an accurate system clock for USB and SPI operation.

Reset and SWD

The NRST pin enables reliable startup and debugging. The SWD interface exposes SWDIO, SWCLK, NRST, 3.3 V and GND for programming with an ST-Link.

GPIO Usage

GPIOs are assigned to SPI, USB, UART, battery sensing, motor outputs, status LED, push button and buzzer, with spare pins reserved for future expansion.

Chapter Summary: *The STM32F411CEU6 acts as the processing core of the Astra Flight Controller, coordinating every subsystem while providing the performance and peripheral support required for stable UAV operation.*

Chapter 5 – ICM-42688-P Inertial Measurement Unit (IMU)

5.1 Introduction

The ICM-42688-P is the primary motion sensing device used in the Astra Flight Controller. It integrates a 3-axis accelerometer and a 3-axis gyroscope in a compact LGA package. The sensor continuously measures aircraft motion and sends this data to the STM32F411CEU6 through the SPI interface for attitude estimation and flight stabilization.

Why the ICM-42688-P was Selected

The sensor was selected because of its excellent gyroscope noise performance, high sampling rate, low latency, low power consumption, compact size, and broad support in modern flight-control firmware such as Betaflight and INAV.

Main Specifications

Parameter	Specification
Manufacturer	TDK InvenSense
Device	ICM-42688-P
Sensor Type	6-Axis IMU
Accelerometer	3-Axis MEMS
Gyroscope	3-Axis MEMS
Operating Voltage	1.71–3.6 V
Communication	SPI / I ² C
Package	LGA-14 (2.5 × 3.0 mm)
Max SPI Clock	Up to 24 MHz
Operating Temp.	–40°C to +85°C

Working Principle

The accelerometer measures linear acceleration along the X, Y, and Z axes, while the gyroscope measures angular velocity about the same axes. Together they provide six degrees of freedom (6-DoF), allowing the firmware to estimate the aircraft's orientation using sensor-fusion algorithms.

Hardware Integration

The IMU operates from the regulated 3.3 V rail. VDD and VDDIO are connected to 3.3 V, while the GND pins connect directly to the ground plane. Local decoupling capacitors are placed close to the power pins to reduce supply noise and improve stability.

SPI Communication

Communication with the STM32F411CEU6 takes place over SPI using SCK, MOSI, MISO, and CS. SPI was selected instead of I²C because it offers higher bandwidth, lower latency, and improved reliability in electrically noisy environments.

Interrupt Output

The interrupt pin notifies the STM32 whenever new sensor data is available, reducing processor workload and improving the timing of the flight-control loop.

PCB Design Considerations

The IMU is placed close to the centre of the PCB, away from the switching regulator and high-current traces. SPI traces are kept short, routed together, and referenced to a continuous ground plane. High-current routing beneath the IMU is avoided to minimise electrical noise.

System Operation

The IMU continuously measures aircraft motion, sends accelerometer and gyroscope data to the STM32 through SPI, and the firmware processes this information to estimate attitude and generate motor control outputs.

Chapter Summary: *The ICM-42688-P provides accurate motion sensing for the Astra Flight Controller. Proper power supply design, PCB placement, grounding, and SPI routing ensure reliable sensor performance and stable flight operation.*

Chapter 6 – USB Type-C Interface

6.1 Introduction

The USB Type-C interface serves as one of the primary communication and power interfaces of the Astra Flight Controller. It provides firmware flashing, debugging, configuration, diagnostics and development power.

USB Type-C offers reversible insertion, improved durability, better current capability and long-term compatibility. The design uses the USB 2.0 Full-Speed interface connected directly to the STM32F411CEU6.

Purpose

Firmware upload, configuration using Betaflight/INAV/STM32CubeProgrammer, debugging through serial communication, and powering the board during development without a LiPo battery.

Advantages

Reversible connector, stronger mechanical retention, higher current capability and modern industry-standard compatibility.

6.2 USB Connector

Pin	Function
VBUS	+5V
GND	Ground
D+	USB Data Positive
D-	USB Data Negative
CC1	Configuration Channel 1
CC2	Configuration Channel 2
Shield	Connector Shield

USB 3.x SuperSpeed pins are unused.

VBUS and Ground

VBUS powers the board during development while the battery remains the primary source during flight. Ground pins and shield connect to the main ground plane for signal integrity and EMI reduction.

6.3 Data Communication

USB Full-Speed uses the D+ and D- differential pair. Route them together with matched lengths, minimal vias, constant spacing, no sharp bends and a continuous ground plane.

The STM32F411CEU6 supports USB 2.0 Full-Speed (12 Mbps), sufficient for firmware updates and configuration.

6.4 Configuration Channel

CC1 and CC2 determine cable orientation and device role. The Astra Flight Controller uses 5.1 kΩ pull-down resistors on both CC pins to identify itself as a USB device and enable 5V from the host.

6.5 PCB Design

Place the connector on the board edge. Keep USB traces away from the buck converter and high-current paths. Future revisions should include a dedicated ESD protection IC.

Chapter Summary: Proper implementation of USB routing, CC resistors, grounding and connector placement ensures reliable programming, debugging and communication.

Chapter 7 – External Interfaces and Connectors

7.1 Overview

The external interfaces connect the Astra Flight Controller to power sources, programming tools, ESCs and external peripherals. Each connector is selected to simplify assembly, debugging and future expansion while maintaining reliable electrical connections.

7.2 XT30 Battery Connector

The XT30 connector is the primary battery input. Battery voltage enters the board through the XT30 connector, passes through the resettable fuse and protection circuitry, and then feeds the MP2359DJ buck converter. Wide copper traces are used to carry battery current and minimise voltage drop.

7.3 SWD Programming Header

Signal	Purpose
SWDIO	Debug Data
SWCLK	Debug Clock
NRST	Reset
3.3 V	Target Reference
GND	Ground

The SWD interface is used with an ST-Link programmer for firmware uploading, debugging and recovery.

7.4 ESC Connections

Motor control signals generated by the STM32 are routed to the ESC outputs. These connections carry logic-level control signals only; motor power is supplied separately by the battery through the ESCs.

7.5 UART Expansion

UART headers allow connection of peripherals such as GPS modules, telemetry radios, Bluetooth adapters or other serial devices. Keeping UART interfaces accessible makes future upgrades straightforward.

7.6 User Interface

The board includes a status LED, a push button and an active buzzer. The LED provides visual status indication, the push button can be used for user input or boot functions, and the buzzer provides audible alerts for arming, warnings and fault conditions.

7.7 PCB Design Notes

All connectors are placed near the board edge for easy access. High-current connectors are routed with wide traces, while communication connectors are kept away from switching power circuitry to reduce electrical noise.

Chapter Summary: *The external interfaces provide reliable access to power, programming, motor outputs and expansion peripherals. Proper connector placement and routing improve manufacturability, maintenance and future expandability.*

Chapter 8 – Indicators and User Interface

8.1 Overview

The Astra Flight Controller includes a status LED, a TL3342 push button and an active buzzer. These devices provide visual, audible and manual interaction during development, testing and flight operation.

8.2 Status LED

The STM32 controls the LED to indicate power, boot status, firmware activity, calibration, arming state and error conditions.

State	Meaning
Solid ON	Board powered
Slow Blink	Initialization
Fast Blink	Firmware activity
Pattern	Warning/Error

8.3 Push Button

The TL3342 tactile switch can be used for boot mode selection, firmware recovery, calibration and user-defined functions through a GPIO input.

8.4 Active Buzzer

The active buzzer provides audible feedback including startup, arming, low-battery and fault indications. It requires only an on/off control signal from the STM32.

8.5 PCB Design Considerations

The LED is placed for visibility, the push button is accessible from the board edge, and the buzzer is positioned to maximise sound output while remaining away from sensitive circuitry.

Chapter Summary: *These user-interface components improve usability, diagnostics and maintenance by providing clear visual, audible and manual interaction with the Astra Flight Controller.*

Chapter 9 – PCB Layout & Design Considerations

9.1 Introduction

9.1.1 Overview

The PCB layout is one of the most critical aspects of the Astra Flight Controller. While the schematic defines how components are electrically connected, the PCB layout determines how reliably the circuit performs in real-world conditions. A poorly routed PCB can introduce electrical noise, voltage drops, communication errors, overheating, and unstable sensor readings, even if the schematic is electrically correct.

The Astra Flight Controller has been designed as a compact 2-layer PCB suitable for fabrication through JLCPCB, while maintaining a board size within the 100 mm × 100 mm manufacturing limit. Throughout the layout process, emphasis was placed on minimizing electrical noise, shortening critical signal paths, and ensuring that sensitive analog components remain isolated from high-current switching circuitry.

The layout was created using KiCad 10, following common PCB design practices recommended by STMicroelectronics, TDK InvenSense, and Monolithic Power Systems.

9.1.2 PCB Design Objectives

The PCB layout was designed to satisfy several engineering objectives:

- Reliable operation under electrical and mechanical stress.
- Low electrical noise around the IMU.
- Efficient power distribution.
- Simple manufacturing using standard PCB fabrication services.
- Easy assembly and inspection.
- Good thermal performance.
- Room for future hardware expansion.

9.2 Board Dimensions

The Astra Flight Controller PCB was intentionally designed to remain within the 100 mm × 100 mm manufacturing limit offered by JLCPCB's standard service. This provides several advantages: lower manufacturing cost, faster production, standard fabrication process, and greater compatibility with PCB assembly services.

Although the actual flight controller occupies only a small portion of the available area, designing within this limit allows flexibility for mounting holes, connectors, and future revisions.

9.3 Component Placement Strategy

9.3.1 General Philosophy

Component placement directly affects signal integrity, thermal performance, manufacturability, and overall reliability. Rather than placing components wherever space was available, the PCB was organized into functional blocks:

- Power input section
- Voltage regulation section
- Microcontroller section

- IMU section
- USB interface
- User interface
- Programming interface
- Expansion connectors

Grouping related components reduces routing complexity and simplifies troubleshooting.

9.3.2 Microcontroller Placement

The STM32F411CEU6 is positioned near the center of the PCB. This location minimizes the average distance between the MCU and all connected peripherals, resulting in shorter SPI traces, shorter USB connections, reduced propagation delay, easier routing, and improved signal integrity.

The exposed thermal pad beneath the MCU is connected directly to the ground plane for both thermal dissipation and electrical grounding.

9.3.3 IMU Placement

The ICM-42688-P is the most sensitive component on the board. To maximize measurement accuracy, it is placed close to the center of the PCB, kept away from the MP2359DJ buck converter and inductor, and high-current traces are not routed beneath the sensor. A continuous ground plane is maintained underneath the device, and the sensor orientation matches the coordinate system expected by the firmware.

Proper placement reduces electrical interference and vibration-induced measurement errors.

9.3.4 Power Section Placement

The power conversion circuitry is grouped together near the battery input. This includes:

- XT30 connector.
- Resettable fuse.
- Protection diode.
- MP2359DJ buck converter.
- 4.7 μ H Bourns SRP5030TA-4R7M inductor.
- Input and output capacitors.
- AP2112K voltage regulator.

Keeping these components close together minimizes loop area and improves power conversion efficiency.

9.3.5 USB Interface Placement

The USB Type-C connector is located on the edge of the PCB. This provides easy cable access, improved mechanical support, short USB data traces, and simplified assembly. The CC resistors and supporting passive components are placed close to the connector to reduce routing complexity.

9.4 Routing Strategy

9.4.1 Signal Routing

Signal traces were routed according to the following guidelines:

- Keep traces as short as practical.
- Avoid unnecessary vias.
- Minimize sharp 90° corners by using 45° bends where possible.

- Route related signals together.
- Keep digital and power traces separated where practical.
- Avoid long parallel runs that may introduce crosstalk.

9.4.2 Power Routing

Power traces carry significantly higher current than logic signals and therefore require greater trace widths. Special attention was given to battery input traces, the buck converter switching loop, 5 V distribution, 3.3 V distribution, and ground return paths.

9.4.3 Ground Plane

A continuous ground plane is one of the most important features of the PCB, providing low-impedance return paths, reduced electromagnetic interference, improved thermal dissipation, better signal integrity, and reduced voltage fluctuations. Where possible, the ground plane remains uninterrupted beneath digital and analog circuitry.

9.5 Power Distribution Network (PDN)

9.5.1 Introduction

A Power Distribution Network (PDN) is responsible for delivering stable voltage to every component on the PCB. Even the best voltage regulator cannot guarantee proper operation if the PCB layout introduces excessive resistance, inductance, or voltage drops.

The PDN was designed with three primary objectives: deliver stable voltages under varying load conditions, minimize voltage ripple and switching noise, and ensure that high-current and low-current circuits do not interfere with one another.

The board contains three primary power rails: VBAT (direct battery voltage from the XT30 connector), +5 V (generated by the MP2359DJ buck converter), and +3.3 V (generated by the AP2112K low-dropout regulator).

9.5.2 High-Current Routing

The battery input and switching regulator carry the highest current on the PCB. Design rules applied include:

- Battery traces are made significantly wider than signal traces.
- The distance between the XT30 connector and the buck converter is minimized.
- Switching current loops are kept as small as possible.
- Copper bottlenecks are avoided.
- Multiple vias are used when transferring high-current paths between PCB layers.

9.5.3 Low-Current Distribution

After regulation, the 5 V and 3.3 V rails supply low-power digital circuits. Separate power branches feed different functional blocks, long power traces are avoided, sensitive analog circuits are isolated from noisy switching nodes, and each integrated circuit receives its supply through short connections.

9.6 Decoupling Capacitors

9.6.1 Purpose

Every digital integrated circuit rapidly switches internal transistors on and off, causing short bursts of current demand. Decoupling capacitors act as local energy reservoirs, supplying these transient current demands

instantly, reducing voltage ripple, improving power stability, lowering EMI, and enabling reliable microcontroller operation and cleaner sensor measurements.

9.6.2 Placement Guidelines

Decoupling capacitors should always be positioned as close as possible to the corresponding power pins. STM32 decoupling capacitors are placed adjacent to each VDD pin, ICM-42688-P power capacitors are located immediately beside the VDD and VDDIO pins, and regulator input/output capacitors are placed directly next to the MP2359DJ and AP2112K.

9.6.3 Bulk Capacitors

In addition to small ceramic decoupling capacitors, larger bulk capacitors stabilize the supply during sudden current changes, reduce low-frequency voltage ripple, improve regulator transient response, and help absorb energy during startup.

9.7 Thermal Management

9.7.1 Heat Sources

Although the Astra Flight Controller has relatively low power consumption, several components generate heat during operation, including the MP2359DJ buck converter, AP2112K regulator, STM32F411CEU6, and the protection diode under high load conditions.

9.7.2 Copper Area

Copper acts as an effective heat spreader. Large copper pours are used around the buck converter, ground copper is connected to exposed thermal pads where applicable, and high-power components are given adequate surrounding copper.

9.7.3 Thermal Vias

Where appropriate, thermal vias connect copper pours on different PCB layers, improving heat transfer and reducing hotspot temperatures. Although the Astra Flight Controller is a two-layer design, strategically placed vias still contribute to heat distribution.

9.8 Electromagnetic Interference (EMI)

9.8.1 Sources of EMI

Potential EMI sources on the Astra Flight Controller include the MP2359DJ switching regulator, high-current battery traces, USB communication lines, and fast digital signals from the STM32.

9.8.2 EMI Reduction Techniques

- Keep switching regulator loops compact.
- Maintain a continuous ground plane.
- Separate high-current traces from sensor signals.
- Route SPI traces away from noisy power circuitry.
- Place decoupling capacitors close to IC power pins.
- Minimize loop areas in power circuits.

9.8.3 Sensor Isolation

The IMU is especially sensitive to electrical noise. To protect measurement accuracy, the IMU is physically separated from the buck converter, no high-current traces pass beneath the sensor, a solid ground plane is maintained below the device, and SPI routing is kept short and direct.

9.9 PCB Assembly Considerations

To simplify manufacturing and assembly, the PCB was designed with standard surface-mount components compatible with JLCPCB assembly services, including clear silkscreen reference designators, proper component orientation, adequate spacing for soldering, standard KiCad footprints, clearly marked polarized components, and accessible test points for debugging.

9.10 Design Verification

9.10.1 Importance of Verification

Completing the schematic and PCB layout does not guarantee that a board is ready for manufacturing. Before fabrication, every design must undergo verification steps to ensure the PCB is electrically correct, mechanically manufacturable, and free from design errors. Verification was carried out using KiCad's built-in validation tools along with manual inspection.

The primary goals of verification were to detect electrical connection errors, verify PCB manufacturing constraints, confirm component footprints, check connector orientations, ensure proper grounding, validate power distribution, and reduce the possibility of assembly errors.

9.10.2 Electrical Rule Check (ERC)

The Electrical Rule Check analyzes the schematic to identify logical or electrical issues before PCB layout or manufacturing. ERC checks include unconnected pins, incorrect pin types, missing power connections, conflicting output signals, floating inputs, duplicate net labels, and invalid power flags.

During development, several ERC warnings were encountered and resolved, including the exposed thermal pad (Pad 49) of the STM32F411CEU6 requiring connection to the ground plane, unused pins requiring No Connect marking, and symbol-to-footprint mismatches.

9.10.3 Design Rule Check (DRC)

The Design Rule Check validates the physical PCB layout against manufacturing capabilities. Typical DRC checks include minimum trace width, minimum spacing between traces, pad-to-pad clearance, via dimensions, hole sizes, copper-to-edge clearance, silkscreen overlap, solder mask violations, and courtyard conflicts.

9.11 PCB Inspection

9.11.1 Visual Inspection

Before exporting manufacturing files, the entire PCB should be inspected manually, checking component orientation, connector positions, silkscreen labels, pin 1 indicators, polarity markings, mounting hole locations, trace routing, ground plane continuity, and copper pours.

9.11.2 3D PCB Inspection

KiCad's three-dimensional PCB viewer allows the complete assembled board to be inspected before manufacturing, helping verify component heights, mechanical clearance, connector accessibility, USB connector orientation, mounting hole placement, and overall board appearance.

9.12 Manufacturing Files

9.12.1 Gerber Files

Gerber files describe the copper, silkscreen, solder mask, and board outline required for PCB fabrication.

Layer	Purpose
Top Copper	Top electrical traces
Bottom Copper	Bottom electrical traces
Top Solder Mask	Protects top copper
Bottom Solder Mask	Protects bottom copper
Top Silkscreen	Component markings
Bottom Silkscreen	Bottom markings
Board Outline	PCB profile

9.12.2 Drill File

The drill file specifies the location and diameter of every drilled hole on the PCB, including through-hole component holes, mounting holes, and via holes.

9.12.3 Bill of Materials (BOM)

The Bill of Materials lists every component required to assemble the flight controller, including reference designator, component value, manufacturer part number, package, and quantity.

9.12.4 Pick-and-Place (CPL) File

For automated assembly, a Pick-and-Place file contains component reference, X coordinate, Y coordinate, rotation angle, and PCB side.

9.13 Manufacturing with JLCPCB

The Astra Flight Controller has been designed to be compatible with JLCPCB's standard PCB fabrication service, considering board dimensions within 100 mm × 100 mm, standard two-layer PCB construction, surface-mount components with commonly available footprints, adequate spacing between components, standard drill sizes, and standard copper thickness.

9.14 Pre-Production Checklist

Schematic

- ERC completed with no critical errors.
- Component values verified.
- Power rails checked.
- Net labels verified.

PCB

- DRC completed successfully.
- Ground plane inspected.
- Differential USB traces verified.
- Power traces checked.
- Silkscreen reviewed.
- Component orientation confirmed.

- Mounting holes verified.

Manufacturing Files

- Gerber files exported.
- Drill files generated.
- BOM completed.
- Pick-and-Place file generated.
- 3D model inspected.

9.15 Final Hardware Validation

After receiving the manufactured PCB, the following validation procedure should be followed before applying full power:

- Visually inspect the PCB for fabrication defects.
- Check for solder bridges and missing components.
- Verify continuity between power and ground.
- Apply USB power first before connecting a LiPo battery.
- Measure the 5 V rail.
- Measure the 3.3 V rail.
- Verify that the STM32 powers up correctly.
- Confirm SWD communication using an ST-Link programmer.
- Upload test firmware.
- Verify IMU communication over SPI.
- Test the status LED, push button, and buzzer.
- Finally, connect the battery and verify full system operation.

Performing these checks in order minimizes the risk of damaging components and simplifies troubleshooting if problems are found.

Chapter Summary: This chapter covered the complete PCB layout and verification process for the Astra Flight Controller, including component placement strategy, routing, power distribution, decoupling, thermal management, EMI reduction, verification (ERC/DRC), inspection, manufacturing file generation, JLCPCB compatibility, and final hardware validation.

Chapter 10 – Power Supply Design

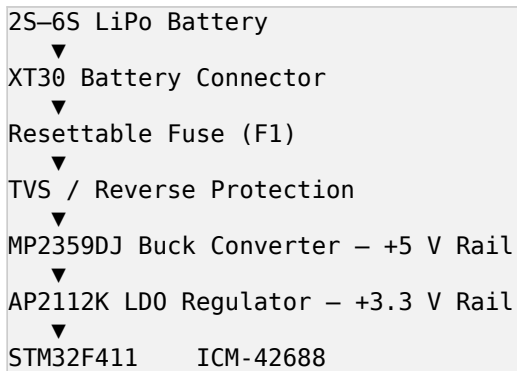
10.1 Introduction

10.1.1 Overview

The power supply is the foundation of every electronic system. Regardless of how powerful the processor or how accurate the sensors are, the entire system depends on receiving a clean, stable, and reliable power source. Even small fluctuations in supply voltage can cause unexpected resets, communication failures, inaccurate sensor readings, or permanent component damage.

The Astra Flight Controller is designed to operate directly from a 2S to 6S Lithium Polymer (LiPo) battery, meaning the input voltage can vary from approximately 7.4 V to 25.2 V. To achieve stable regulation, the Astra Flight Controller uses a two-stage power architecture: the MP2359DJ synchronous buck converter generates a regulated 5 V supply from the battery, and the AP2112K Low Dropout (LDO) regulator generates a clean 3.3 V supply from the 5 V rail.

10.1.2 Power Flow



The two-stage regulation significantly reduces heat generation while ensuring excellent power quality for sensitive digital electronics.

10.2 Battery Input Stage

10.2.1 XT30 Connector

The flight controller receives power through an XT30 connector, selected for its high current capability, low contact resistance, reliable locking mechanism, wide availability in FPV/UAV applications, and excellent vibration resistance.

10.2.2 Supported Battery Range

Battery	Nominal Voltage	Fully Charged
2S	7.4 V	8.4 V
3S	11.1 V	12.6 V
4S	14.8 V	16.8 V
5S	18.5 V	21.0 V
6S	22.2 V	25.2 V

10.2.3 Resettable Fuse (F1)

Immediately after the XT30 connector, the input passes through a resettable polyfuse (PTC), which protects the PCB from excessive current caused by short circuits, incorrect wiring, component failures, or assembly mistakes. Unlike a traditional fuse, the polyfuse automatically resets after the fault is removed and the device cools down.

10.2.4 Input Filtering

Before entering the buck converter, the battery supply is filtered using input capacitors placed close to the MP2359DJ. These capacitors reduce voltage ripple, supply instantaneous current during switching, stabilize the converter input, and improve electromagnetic compatibility (EMC).

10.3 MP2359DJ Buck Converter

10.3.1 Why a Buck Converter?

A linear regulator would dissipate the voltage difference between the battery and the output as heat. Converting 25.2 V from a fully charged 6S battery directly to 5 V using a linear regulator would result in excessive power loss and unacceptable temperatures. A switching buck converter solves this problem by rapidly switching the input voltage through an inductor and filtering the output, achieving efficiencies typically above 85–90%.

10.3.2 MP2359DJ Features

- Wide input voltage range.
- High conversion efficiency.
- Integrated MOSFETs.
- Compact package.
- High switching frequency.
- Internal protection features.
- Minimal external components.

10.3.3 Operating Principle

The MP2359DJ is a synchronous step-down (buck) switching regulator. Its operation consists of four basic stages: the internal MOSFET switches ON allowing current to flow into the inductor; energy is stored in the magnetic field of the inductor; the MOSFET switches OFF; and the inductor releases stored energy through the output filter, maintaining a continuous output voltage. This cycle repeats hundreds of thousands of times per second.

10.3.4 External Components

- Input capacitor.
- Output capacitor.
- Bootstrap capacitor.
- Schottky protection diode (where applicable).
- Feedback resistor network.
- 4.7 μ H power inductor.

10.3.5 Selected Inductor

The Astra Flight Controller uses the Bourns SRP5030TA-4R7M, with an inductance of 4.7 μ H, saturation current greater than 3 A, shielded construction, low DC resistance, and a compact SMD package. The shielded design

helps reduce electromagnetic interference while providing stable operation at the switching frequency of the regulator.

10.4 Feedback Network

The output voltage of the MP2359DJ is determined by a resistor divider connected between the output voltage and the feedback pin. This network continuously monitors the output voltage and adjusts the converter duty cycle to maintain a stable 5 V output despite changes in battery voltage or load current.

10.5 AP2112K Low Dropout Regulator

10.5.1 Purpose

Although the 5 V rail generated by the buck converter is highly efficient, switching regulators inherently produce a small amount of high-frequency ripple. The Astra Flight Controller uses an AP2112K LDO to generate a regulated 3.3 V rail, acting as a secondary filtering stage that significantly reduces switching noise before it reaches the microcontroller and IMU.

10.5.2 Advantages of the AP2112K

- Low output noise.
- Good transient response.
- Stable operation with ceramic capacitors.
- Compact footprint.
- Adequate output current for the flight controller.

Chapter Summary: *The Astra Flight Controller uses a robust two-stage power architecture that converts a 2S–6S LiPo battery into regulated 5 V and 3.3 V supplies. The MP2359DJ buck converter provides efficient primary voltage conversion with minimal heat generation, while the AP2112K low-dropout regulator delivers a clean and stable 3.3 V rail for the STM32 microcontroller and ICM-42688-P IMU.*

Chapter 11 – Firmware & Programming

11.1 Introduction

11.1.1 Overview

The firmware is the software that transforms the Astra Flight Controller from a collection of electronic components into a fully functional embedded control system. While the hardware provides the computational resources, sensors, and communication interfaces, it is the firmware that coordinates these resources to stabilize the aircraft, interpret pilot commands, and manage peripheral devices.

The firmware runs entirely on the STM32F411CEU6, a 32-bit ARM Cortex-M4F microcontroller. Every operation performed by the flight controller—reading sensor data, calculating aircraft attitude, executing control algorithms, generating ESC outputs, monitoring battery voltage, communicating over USB, and driving the user interface—is controlled by the firmware.

Embedded firmware must operate under strict real-time constraints. Sensor data must be processed within microseconds, control outputs must be generated at precise intervals, and communication peripherals must respond without introducing delays that could compromise flight stability. The firmware follows a modular design philosophy, with each subsystem implemented as an independent software module.

11.1.2 Objectives of the Firmware

- Initialize all onboard hardware correctly during startup.
- Configure the system clock and peripheral interfaces.
- Read sensor data from the IMU.
- Calculate aircraft orientation.
- Execute flight control algorithms.
- Generate motor control outputs.
- Monitor system voltage and health.
- Provide communication through USB and UART.
- Control the LED and buzzer.
- Detect and recover from fault conditions.

11.1.3 Software Architecture

Rather than placing all program logic into a single source file, the firmware is organized into separate modules, each responsible for a specific subsystem. A typical directory structure is shown below:

```
Firmware/
├── Core/           (main.c, startup, system clock, IRQ handlers)
├── Drivers/        (GPIO, SPI, UART, USB, TIM, ADC)
├── Sensors/        (ICM42688 driver)
├── FlightControl/  (PID, Mixer, SensorFusion)
├── Peripherals/    (LED, Buzzer, Button)
└── Utilities/
```

Separating the firmware into modules improves readability, reduces debugging time, and allows new features to be added without affecting unrelated code.

11.2 STM32 Boot Process

11.2.1 What Happens When Power Is Applied?

When power is first applied, the STM32F411CEU6 begins executing a predefined startup sequence: power becomes stable, internal reset circuitry is released, the processor begins executing the reset vector, the startup assembly code initializes the system, memory sections are prepared, system clocks are configured, peripheral initialization begins, and finally `main()` is executed.

11.2.2 Reset Sources

- Initial power application.
- Manual reset using the NRST pin.
- Watchdog timer reset.
- Brown-out reset caused by low supply voltage.
- Software-generated reset.
- Debug reset from the ST-Link programmer.

11.2.3 Reset Vector

Every ARM Cortex-M processor begins execution from a predefined location in Flash memory known as the vector table.

Address	Purpose
0x00000000	Initial Stack Pointer
0x00000004	Reset Handler Address

The Reset Handler is the first function executed after every reset, responsible for initializing memory, configuring the processor, preparing the runtime environment, and calling the main application.

11.3 Boot Modes

11.3.1 Boot Configuration

Boot Mode	Description
User Flash Memory	Normal firmware execution
System Memory	Factory bootloader
SRAM	Debugging and testing

Under normal operation, the Astra Flight Controller always boots from internal Flash memory where the flight firmware is stored.

11.3.2 Internal Bootloader

The STM32 contains a factory-programmed bootloader stored in protected system memory that cannot be erased by the user. It provides firmware programming capabilities and serves as a recovery mechanism if the application firmware becomes corrupted.

11.4 Memory Organization

11.4.1 Memory Architecture

Memory	Size	Purpose
Flash Memory	512 KB	Firmware storage
SRAM	128 KB	Runtime variables and stack
System Memory	Internal	Factory bootloader
Peripheral Registers	Memory-mapped	Hardware control

11.4.2 Flash Memory

Flash memory is non-volatile, meaning its contents remain intact even after power is removed. Firmware stored in Flash includes startup code, hardware initialization, peripheral drivers, flight control algorithms, sensor drivers, and communication protocols.

11.4.3 SRAM

SRAM stores data that changes while the program is running, including global variables, sensor measurements, PID controller variables, communication buffers, and stack/heap memory. Unlike Flash, SRAM is cleared whenever power is removed.

11.4.4 Stack and Heap

The stack stores local variables, function parameters, return addresses, and interrupt context for every function call. The heap is used for dynamic memory allocation, which the Astra firmware largely avoids in favor of statically allocated memory to prevent fragmentation and improve predictability.

11.5 Interrupt System

11.5.1 Introduction

The STM32F411 uses the Nested Vectored Interrupt Controller (NVIC) to manage interrupt requests from peripherals such as USB events, SPI data reception, UART communication, timer overflows, IMU data-ready signals, and ADC conversion completion.

11.5.2 Interrupt Priority

Priority	Example
Highest	IMU Data Ready
High	Timer Update
Medium	USB Communication
Lower	UART Reception
Lowest	User Button

11.6 System Clock

11.6.1 Importance of the System Clock

Every operation performed by the microcontroller is synchronized by the system clock, which determines CPU execution speed, timer accuracy, USB communication timing, SPI data rates, UART baud rates, and ADC conversion timing.

11.6.2 External Crystal Oscillator

The Astra Flight Controller uses an external crystal oscillator to provide a stable and accurate reference clock, offering higher frequency accuracy, better temperature stability, reliable USB timing, and more consistent peripheral operation than the internal RC oscillator.

Chapter Summary: This section introduced the firmware architecture, STM32 startup sequence, boot process, memory organization, interrupt system, and system clock — the foundation upon which all higher-level firmware functions are built.

11.7 Peripheral Drivers

11.7.1 Introduction

A peripheral driver is a software module responsible for configuring and controlling a specific hardware peripheral inside the STM32. Instead of directly manipulating hardware registers throughout the program, each peripheral is accessed through a dedicated driver, improving readability and simplifying debugging.

- GPIO
- SPI
- USB
- UART
- I²C
- Timers
- PWM
- ADC
- DMA (future implementation)

11.8 GPIO Driver

11.8.1 Overview

GPIO pins provide the interface between the STM32 and external hardware. Every LED, button, buzzer, sensor interrupt, and communication signal ultimately connects to a GPIO pin, which can operate in Digital Input, Digital Output, Alternate Function, or Analog Input modes.

11.8.2 GPIO Initialization

During firmware startup, each required GPIO port is enabled through the RCC module, after which the firmware configures pin mode, output type, output speed, pull-up/pull-down resistors, and alternate function.

11.8.3 GPIO Usage in Astra Flight Controller

Device	GPIO Function
Status LED	Digital Output
Push Button	Digital Input
Active Buzzer	Digital Output
SPI Signals	Alternate Function
USB D+ / D-	Alternate Function
UART TX/RX	Alternate Function
IMU Interrupt	External Interrupt (EXTI)

11.9 SPI Driver

11.9.1 Why SPI?

The ICM-42688-P communicates with the STM32 using SPI, selected because it offers high communication speed, full-duplex communication, low protocol overhead, excellent reliability, and simple hardware implementation.

11.9.2 SPI Signals

Signal	Function
MOSI	Master Out, Slave In

Signal	Function
MISO	Master In, Slave Out
SCK	Serial Clock
CS	Chip Select

11.9.3 SPI Initialization

The SPI peripheral is configured during startup with master mode, clock polarity (CPOL), clock phase (CPHA), 8-bit data frame size, baud rate prescaler, and software-controlled Chip Select, matching the IMU datasheet requirements.

11.9.4 SPI Data Transfer

A typical SPI transaction pulls Chip Select low, sends the register address, transmits or receives the required data bytes, then pulls Chip Select high. This sequence repeats whenever the firmware reads sensor data or writes configuration registers.

11.10 UART Driver

11.10.1 Purpose

UART provides serial communication between the flight controller and external devices such as a GPS receiver, telemetry radio, Bluetooth module, external computer, or debug console.

11.10.2 UART Configuration

During initialization, the firmware configures baud rate, data length, stop bits, parity, and hardware flow control. Typical baud rates include 9600, 57600, and 115200 bps.

11.10.3 UART Applications

- GPS position updates
- Flight telemetry
- Debug messages
- Firmware diagnostics
- External configuration tools

11.11 USB Driver

11.11.1 Overview

The USB peripheral allows direct communication between the Astra Flight Controller and a host computer. The STM32F411 contains an integrated USB 2.0 Full-Speed controller, eliminating the need for an external USB-to-Serial converter.

11.11.2 USB Initialization

- Enable USB peripheral clock
- Configure D+ and D- pins
- Initialize USB descriptors
- Configure communication endpoints
- Register interrupt handlers
- Enable USB interrupts

11.11.3 USB Applications

- Firmware flashing
- Configuration software
- Debug communication
- Diagnostic data
- Future firmware updates

11.12 I²C Driver

11.12.1 Purpose

Although the current hardware does not use I²C peripherals, the firmware architecture includes support for future expansion such as magnetometers, barometers, EEPROM memory, OLED displays, or environmental sensors.

11.12.2 I²C Characteristics

Compared to SPI, I²C uses only two communication wires and supports multiple devices on one bus with simple wiring, but offers lower speed, shared bus bandwidth, and a slightly more complex protocol.

11.13 Timer Driver

11.13.1 Purpose

Timers generate accurate timing events used for flight control loop timing, PWM generation, time measurements, delays, and scheduling periodic tasks.

11.13.2 Timer Configuration

Each timer is configured with clock source, prescaler, auto-reload value, counter mode, and interrupt enable settings.

11.14 PWM Generation

11.14.1 Introduction

Pulse Width Modulation (PWM) is used to control the ESCs, which in turn regulate motor speed. Each PWM signal consists of a repeating waveform whose pulse width determines the desired motor output.

11.14.2 ESC Output

The firmware continuously updates PWM duty cycles based on the flight control algorithm, combining pilot throttle input, PID controller corrections, sensor measurements, and flight mode.

11.15 ADC Driver

11.15.1 Purpose

The ADC converts analog voltages into digital values, including battery voltage, current sensor output, temperature sensor readings, and external analog inputs.

11.15.2 ADC Operation

The ADC periodically samples the selected analog channel, and the resulting digital value is converted into the corresponding voltage using calibration constants stored in the firmware.

11.16 DMA (Future Expansion)

Direct Memory Access allows data to be transferred between peripherals and memory without continuous CPU involvement. Future firmware versions may use DMA for SPI sensor data acquisition, UART communication, ADC conversions, and USB transfers, reducing CPU workload and improving real-time performance.

Chapter Summary: *This section introduced the peripheral drivers that allow the STM32F411CEU6 to interact with the hardware, covering GPIO, SPI, UART, USB, I²C, timers, PWM, ADC, and DMA — the hardware abstraction layer upon which the higher-level flight control firmware is built.*

11.17 IMU Driver

11.17.1 Introduction

The ICM-42688-P is the primary motion sensor of the Astra Flight Controller. The firmware communicates with the IMU through the SPI interface, retrieving new acceleration and angular velocity measurements every control cycle.

11.17.2 Responsibilities of the IMU Driver

- Detecting the IMU during startup.
- Resetting the sensor.
- Configuring operating modes.
- Selecting accelerometer range.
- Selecting gyroscope range.
- Configuring output data rate.
- Reading sensor measurements.
- Monitoring sensor status.
- Handling communication errors.
- Providing calibrated sensor data to the flight controller.

11.18 IMU Initialization

11.18.1 Startup Sequence

A typical initialization sequence is: enable SPI peripheral, wait for sensor power stabilization, read the WHO_AM_I register, verify the returned device ID, perform a software reset, wait for reset completion, configure the internal clock, configure accelerometer and gyroscope settings, configure digital filters, enable measurement mode, and verify configuration registers.

11.18.2 Device Identification

Before configuring the sensor, the firmware reads the WHO_AM_I register to verify communication. If the value is incorrect, the firmware assumes incorrect SPI wiring, an unpowered IMU, the wrong device connected, or communication failure, and indicates the error via the status LED or buzzer.

11.19 Accelerometer Configuration

11.19.1 Purpose

The accelerometer measures linear acceleration along the X, Y, and Z axes, used for determining aircraft orientation relative to gravity, detecting rapid movement, estimating tilt angles, and assisting sensor fusion algorithms.

11.19.2 Measurement Range

Range	Characteristic
± 2 g	Highest sensitivity
± 4 g	Moderate sensitivity
± 8 g	Higher dynamic range
± 16 g	Maximum dynamic range

11.19.3 Output Data Rate

The Output Data Rate (ODR) determines how frequently the accelerometer generates new measurements. Higher ODR values provide faster response and lower latency, but increase power consumption, processor workload, and communication bandwidth requirements.

11.20 Gyroscope Configuration

11.20.1 Purpose

The gyroscope measures angular velocity, indicating roll, pitch, and yaw rate, and is the primary sensor used for maintaining stable flight.

11.20.2 Measurement Range

Range	Typical Application
$\pm 250^\circ/\text{s}$	Slow rotation
$\pm 500^\circ/\text{s}$	General-purpose control
$\pm 1000^\circ/\text{s}$	Fast maneuvers
$\pm 2000^\circ/\text{s}$	Racing drones and acrobatics

11.20.3 Digital Low-Pass Filter

Programmable digital filters reduce high-frequency noise, producing smoother measurements, reduced vibration effects, improved PID stability, and better attitude estimation. Excessive filtering increases latency, while insufficient filtering allows vibration noise to reach the control algorithm.

11.21 SPI Register Communication

11.21.1 Register Access

The firmware communicates with the IMU by reading and writing internal registers controlling power management, sensor configuration, data output, interrupt settings, and FIFO operation.

11.21.2 Reading Registers

A typical register read asserts Chip Select low, sends the register address with the read bit set, receives the requested data bytes, and releases Chip Select.

11.21.3 Writing Registers

Configuration registers are written by pulling Chip Select low, sending the register address, sending the configuration value, and releasing Chip Select. These are typically written only during initialization.

11.22 Reading Motion Data

11.22.1 Data Registers

The accelerometer and gyroscope store measurements in dedicated output registers, each consisting of a high byte and a low byte combined into a signed 16-bit integer.

11.22.2 Unit Conversion

Raw sensor values are converted into engineering units (acceleration in g or m/s^2 , angular velocity in $^\circ/\text{s}$ or rad/s) using the sensitivity values specified by the selected measurement range.

11.23 FIFO Buffer

11.23.1 Purpose

The ICM-42688-P includes an internal FIFO buffer that temporarily stores multiple samples, reducing SPI traffic, lowering CPU utilization, improving sensor synchronization, and reducing the likelihood of missing samples during periods of high processor load.

11.23.2 FIFO Operation

When enabled, sensor measurements are written into the FIFO automatically, the STM32 periodically reads several samples in one SPI transaction, and processed samples are removed from the FIFO. Burst transfers improve communication efficiency compared to reading each sample individually.

11.24 Sensor Data Validation

Before using sensor measurements, the firmware verifies successful SPI communication, detects invalid register values, monitors FIFO overflow conditions, confirms sensor status flags, checks for unrealistic measurements, and detects repeated or stale data. If an error is detected, the firmware may reinitialize the IMU, ignore corrupted samples, notify the user, or enter a safe operating state.

Chapter Summary: This section described the firmware responsible for communicating with the ICM-42688-P IMU, covering initialization, device identification, accelerometer and gyroscope configuration, SPI register access, measurement retrieval, unit conversion, FIFO buffering, and sensor data validation.

11.32 Bootloader

11.32.1 Introduction

A bootloader is a small program that executes immediately after the microcontroller resets, preparing the processor for running the main firmware or allowing new firmware to be uploaded. The STM32F411CEU6 contains a factory-programmed bootloader stored in protected System Memory that cannot be erased or modified by the user, serving as a reliable recovery mechanism if the application firmware becomes corrupted.

11.32.2 Boot Sequence

```
Power Applied → Hardware Reset → BOOT Configuration Checked
→ Select Boot Device → Load Vector Table → Run Reset Handler
→ Initialize Hardware → Start main()
```

11.32.3 Boot Options

Boot Source	Purpose
User Flash	Normal flight firmware
System Memory	Factory bootloader
SRAM	Debugging and testing

11.33 SWD Programming Interface

11.33.1 Overview

The primary programming and debugging interface used by the Astra Flight Controller is Serial Wire Debug (SWD), a two-wire debugging interface developed by ARM as a simplified alternative to JTAG, requiring only two communication signals while supporting real-time debugging, Flash programming, memory inspection, and breakpoint debugging.

11.33.2 SWD Signals

Signal	Purpose
SWDIO	Bidirectional data line
SWCLK	Debug clock
NRST	Hardware reset
3.3V	Target voltage reference
GND	Ground reference

11.33.3 ST-Link Programmer

The Astra Flight Controller is designed to be programmed using an ST-Link V2 or ST-Link V3 programmer, providing Flash programming, debugging, single-step execution, register inspection, memory access, and breakpoint control. Because SWD communicates directly with the processor, it remains functional even if the main firmware is corrupted.

11.34 Firmware Flashing

11.34.1 Programming Procedure

- Connect the ST-Link to the SWD header.
- Power the flight controller.
- Open STM32CubeProgrammer or the preferred IDE.
- Detect the target device.
- Erase Flash memory if necessary.
- Upload the compiled firmware.
- Verify the programmed memory.
- Reset the processor.
- Observe startup behavior.

11.34.2 Flash Verification

After programming, the debugger compares the contents of Flash memory with the firmware image, confirming successful programming, no corrupted Flash sectors, correct firmware version, and valid memory contents.

11.35 Debugging

11.35.1 Why Debugging Matters

Even carefully written firmware may contain errors such as incorrect peripheral configuration, infinite loops, communication failures, incorrect interrupt priorities, timing errors, or memory corruption. Early debugging greatly reduces development time.

11.35.2 Breakpoints

A breakpoint pauses program execution at a specific instruction, allowing the developer to inspect variable values, CPU registers, memory contents, peripheral registers, and the program counter.

11.35.3 Single-Step Execution

Instead of running continuously, the processor can execute one instruction at a time, helping developers understand function execution order, register changes, peripheral initialization, and interrupt behavior.

11.36 Fault Handling

11.36.1 Types of Faults

Fault	Description
Hard Fault	Serious processor exception
Memory Management Fault	Invalid memory access
Bus Fault	Communication with memory failed
Usage Fault	Illegal instruction or invalid operation

11.36.2 Hard Fault Handler

When a Hard Fault occurs, execution immediately transfers to the `HardFault_Handler()` function, which stops motor outputs, disables interrupts, records diagnostic information, activates the status LED, sounds the buzzer, and waits for reset.

11.37 Watchdog Timer

11.37.1 Purpose

A watchdog timer improves system reliability by automatically resetting the microcontroller if the firmware stops executing correctly, due to infinite loops, deadlocks, unexpected software crashes, or memory corruption.

11.37.2 Independent Watchdog (IWDG)

The STM32F411 includes an Independent Watchdog that operates from its own internal clock source, continuing to operate even if the main clock fails, independent of application software, and improving overall system reliability.

11.38 Logging and Diagnostics

During firmware development, diagnostic messages transmitted over USB or UART typically include startup status, IMU initialization results, sensor values, loop execution time, battery voltage, error codes, and firmware version.

11.39 Firmware Update Strategy

Future firmware releases may introduce improved flight algorithms, additional sensor support, new communication protocols, performance optimizations, and bug fixes. Firmware versions should be clearly

numbered, release notes should accompany each version, existing configuration data should be preserved where possible, and updates should always be verified after programming.

Chapter Summary: This section covered the mechanisms used to program, debug, and maintain the firmware, including the STM32 bootloader, SWD interface, firmware flashing, debugging techniques, fault handling, watchdog timers, and diagnostic logging.

11.40 Firmware Architecture

11.40.1 Introduction

The Astra Flight Controller follows a layered and modular architecture, where each software component performs a specific task and communicates with other modules through well-defined interfaces, reducing dependencies and minimizing the impact of future changes.

11.40.2 Firmware Layers

```
Flight Control Algorithms (PID, Sensor Fusion, Mixer)
Application Layer (Battery, LEDs, Buzzer, USB)
Device Drivers (SPI, UART, GPIO, TIM, ADC)
STM32 HAL / LL Libraries
STM32F411 Hardware
```

Each layer depends only on the layer directly below it, making replacing or upgrading individual modules much easier.

11.41 Project Organization

11.41.1 Folder Structure

```
Firmware/
├── Core/           (main.c, interrupt handlers, startup, clock config)
├── Drivers/        (GPIO, SPI, UART, USB, ADC, Timer)
├── Sensors/        (ICM-42688 driver, calibration routines, utilities)
├── FlightControl/  (PID controller, motor mixer, sensor fusion, flight modes, failsafe)
├── Peripherals/    (LED control, push button, buzzer, battery monitor)
├── Utilities/
├── Configuration/
├── Middleware/
└── Documentation/
```

11.42 Main Program Flow

After initialization is complete, execution enters the infinite loop inside `main()`, which never exits:

```
Initialize Hardware → Initialize Drivers → Initialize IMU → Calibrate Sensors
→ Enter Main Loop →
Read Sensors → Estimate Attitude → Read Receiver Inputs → Run PID Controller
→ Motor Mixer → Generate ESC Outputs → Check Battery → USB/UART Tasks → Repeat
```

11.43 Task Scheduler

11.43.1 Why Scheduling Is Needed

Not every task requires the same update frequency. Running every task at the highest frequency would waste processor time, so a scheduler ensures each task executes only as often as necessary.

11.43.2 Example Schedule

Task	Typical Frequency
IMU Read	8 kHz
Sensor Fusion	4 kHz
PID Controller	4 kHz
ESC Output	4 kHz
Battery Monitor	10 Hz
USB Communication	100 Hz
Status LED	2 Hz
Buzzer	As Required

11.44 Configuration Files

Instead of hardcoding every parameter, firmware settings are centralized, including SPI baud rate, PID gains, loop frequency, IMU range, gyroscope filter, battery warning voltage, USB descriptors, and UART baud rates.

11.45 Error Handling

A flight controller must continue operating safely even when unexpected problems occur, such as IMU communication failure, low battery, USB disconnect, sensor timeout, invalid calibration, memory corruption, or unexpected reset. Typical responses include flashing the status LED, activating the buzzer, logging diagnostic information, entering failsafe mode, and preventing motor arming.

11.46 Betaflight Compatibility

The Astra Flight Controller has been designed with hardware capable of supporting Betaflight in future firmware revisions, including the STM32F411 processor, SPI IMU, USB interface, multiple timers, PWM outputs, sufficient Flash memory, and adequate SRAM. Compatibility also requires target configuration, pin mapping, timer mapping, IMU driver configuration, board definition, and build configuration.

11.47 INAV Compatibility

The hardware is also suitable for INAV, particularly for navigation-oriented aircraft, offering GPS navigation, Return-to-Home, waypoint missions, Position Hold, and Cruise Mode. Full INAV support may require a dedicated GPS UART, barometer, magnetometer, larger Flash memory, and external flash for Blackbox logging.

11.48 Code Documentation

Well-written firmware should always include documentation covering function descriptions, parameter explanations, return values, hardware references, register descriptions, version history, and modification dates.

11.49 Version Control

The Astra Flight Controller firmware should be managed using Git, providing complete change history, easy rollback, team collaboration, branch management, issue tracking, and release management.

```
AstraFC/
├── Hardware/
├── Firmware/
├── Documentation/
├── BOM/
├── Gerbers/
├── Libraries/
└── Images/
```


11.50 Future Expansion

The modular firmware architecture allows future enhancements without requiring major redesign:

Flight Features

- Acro Mode
- Angle Mode
- Horizon Mode
- Turtle Mode
- Air Mode

Sensors

- Magnetometer
- Barometer
- GPS
- Optical Flow
- Range Finder

Communication

- MSP Protocol
- CRSF
- SBUS
- SmartPort
- MAVLink
- Bluetooth
- Wi-Fi

Data Logging

- Blackbox logging
- SD card support
- External Flash memory
- USB log export

Safety Features

- Geofencing
- Automatic Return-to-Home
- GPS Rescue
- Dynamic failsafe
- Motor fault detection
- ESC telemetry
- Battery health estimation

Chapter Summary: This section described the overall firmware architecture, project organization, main execution flow, task scheduling, configuration management, error handling, and compatibility with Betaflight and INAV, along with the importance of documentation, version control, and modularity.

11.51 Firmware Optimization

11.51.1 Introduction

One of the defining characteristics of a flight controller is its ability to execute thousands of calculations every second while maintaining deterministic timing. Firmware optimization is about ensuring that all critical tasks complete within their deadlines while maintaining predictable behavior.

11.52 CPU Utilization

11.52.1 Processor Workload

The STM32F411CEU6 contains a 100 MHz ARM Cortex-M4F processor with a hardware FPU. The firmware continuously performs IMU reading, sensor fusion, PID calculations, ESC updates, battery monitoring, USB/UART communication, and LED/buzzer control. A typical operating target is 50–70% CPU utilization, leaving reserve capacity.

11.52.2 Measuring CPU Load

- Measuring loop execution time.
- Using timer peripherals.
- Profiling functions with a debugger.
- Toggling a GPIO pin during execution and observing it with an oscilloscope or logic analyzer.

11.53 Memory Optimization

11.53.1 Flash Memory Usage

The 512 KB of Flash memory is used for startup code, peripheral drivers, flight algorithms, USB stack, sensor drivers, configuration tables, and constant lookup data. Recommended practices include removing unused libraries, avoiding duplicate code, using reusable functions, and storing constant data in Flash instead of SRAM.

11.53.2 SRAM Usage

The 128 KB of SRAM stores sensor measurements, communication buffers, stack, global variables, PID state variables, and flight status information. Excessive RAM usage can reduce available stack space and lead to unpredictable behavior.

11.53.3 Stack Monitoring

Stack overflow is one of the most common causes of embedded firmware crashes. To reduce risk: avoid unnecessarily large local arrays, minimize recursive functions, prefer static or global allocation for large data structures, and monitor stack usage during debugging.

11.54 Floating-Point Optimization

11.54.1 Floating Point Unit (FPU)

The Cortex-M4F's hardware FPU accelerates sensor fusion, PID control, quaternion mathematics, trigonometric functions, and matrix calculations, significantly improving performance compared to software-based floating-point arithmetic.

11.54.2 Mathematical Efficiency

- Reuse previously calculated values.

- Avoid repeated trigonometric calculations.
- Cache intermediate results.
- Reduce unnecessary floating-point conversions.
- Precompute constant values when possible.

11.55 Interrupt Optimization

11.55.1 Interrupt Design

Interrupts should execute as quickly as possible. An ISR should read or write the required hardware registers, store any necessary data, set a flag indicating processing is required, and return immediately. Lengthy computations inside interrupts increase interrupt latency.

11.55.2 Interrupt Priorities

Priority Level	Assigned Task
Highest	IMU Data Ready
High	Timer Update
Medium	SPI Transfer Complete / USB Events
Low	UART Reception
Lowest	User Button

11.56 DMA Optimization & Real-Time Constraints

Direct Memory Access (DMA) allows peripherals to transfer data directly to memory without continuous CPU involvement, providing lower CPU utilization, faster data transfers, reduced interrupt frequency, and improved real-time performance. DMA is particularly beneficial for SPI IMU transfers, ADC sampling, UART communication, and USB packet transfers.

Flight controllers are real-time systems: IMU data must be processed before the next sample arrives, PID calculations must finish before motor outputs are updated, and ESC signals must be generated at precise intervals. Jitter — variations in task execution timing — can arise from long interrupt routines, blocking communication functions, variable loop execution time, and memory contention; reducing jitter improves flight control loop consistency.

11.57 Power Optimization & Clock Management

Unused peripherals consume unnecessary power. The firmware disables hardware modules that are not required during operation, such as unused UARTs, unused SPI interfaces, unused ADC channels, and disabled timers, reducing power consumption, electromagnetic interference, and heat generation.

11.58 Code Optimization Best Practices

- Keep functions short and focused.
- Minimize global variables.
- Use meaningful variable names.
- Eliminate unused code.
- Avoid blocking delays.
- Replace repeated code with reusable functions.
- Comment complex algorithms.
- Follow a consistent coding style.

- Validate all user inputs.
- Check return values from peripheral drivers.

11.59 Performance Testing Framework

Before release, firmware performance should be evaluated under realistic operating conditions:

- CPU load measurement.
- Memory usage verification.
- Long-duration stability testing.
- Sensor communication reliability.
- USB communication under load.
- Battery monitoring accuracy.
- Maximum loop frequency.
- Stress testing with continuous operation.

Chapter Summary: *This section focused on optimizing the Astra Flight Controller firmware for reliable real-time performance, covering CPU utilization, memory management, stack monitoring, floating-point optimization, interrupt optimization, DMA, deterministic execution, jitter reduction, power management, coding best practices, and performance testing.*

Chapter 12 – Manufacturing & Assembly

ASTRA FLIGHT CONTROLLER DOCUMENTATION

12.1 Introduction

12.1.1 Overview

Once the PCB design has been completed and verified, the next stage of the Astra Flight Controller project is manufacturing and assembly. This phase transforms the digital design files into a fully functional physical board.

Manufacturing involves fabricating the printed circuit board (PCB), while assembly refers to soldering all electronic components onto the board. Careful attention during these stages is essential, as mistakes such as incorrect component placement, poor solder joints, or manufacturing defects can lead to unreliable operation or complete board failure.

The Astra Flight Controller has been designed with manufacturing in mind, using readily available surface-mount components and a compact PCB layout that fits within 100 mm × 100 mm, making it suitable for low-cost PCB fabrication services such as JLCPCB.

12.1.2 Manufacturing Workflow

The overall manufacturing process consists of the following stages:



Each stage should be completed successfully before moving to the next.

12.2 Design Verification

12.2.1 Electrical Rule Check (ERC)

Before manufacturing begins, the schematic should pass an Electrical Rule Check (ERC). The ERC verifies:

- Missing power connections.
- Floating inputs.
- Incorrect pin types.
- Unconnected nets.
- Conflicting electrical connections.

All critical ERC errors should be resolved before generating manufacturing files. Minor warnings should be reviewed individually to determine whether they are intentional.

12.2.2 Design Rule Check (DRC)

The PCB layout should also pass a Design Rule Check (DRC). The DRC verifies:

- Track widths.
- Copper clearances.
- Via sizes.
- Hole diameters.
- Silk screen overlap.
- Solder mask clearances.
- Board outline integrity.

Passing the DRC significantly reduces the likelihood of fabrication issues.

12.3 Manufacturing Files

12.3.1 Required Files

PCB manufacturers require several files to produce the board. The Astra Flight Controller manufacturing package includes:

File	Purpose
Gerber Files	PCB copper, silkscreen, solder mask
Drill Files	Hole locations
Pick-and-Place File	Automated component placement
Bill of Materials (BOM)	Component information
Assembly Drawing	Component locations
PCB Preview	Visual verification

These files are generated directly from KiCad after the PCB design has been finalized.

12.3.2 Gerber Files

Gerber files describe every physical layer of the PCB. Typical layers include:

- Top Copper
- Bottom Copper

- Top Silkscreen
- Bottom Silkscreen
- Top Solder Mask
- Bottom Solder Mask
- Board Outline

PCB manufacturers use these files to fabricate the board accurately.

12.3.3 Drill Files

Drill files specify the location and diameter of every drilled hole on the PCB. These include:

- Mounting holes.
- Through-hole vias.
- Component mounting holes.

Accurate drill files are essential to ensure correct component fitment and mechanical alignment.

12.4 PCB Specifications

The Astra Flight Controller uses the following PCB specifications:

Parameter	Value
Board Size	≤ 100 mm × 100 mm
Layers	4
PCB Thickness	1.6 mm
Copper Weight	1 oz
Surface Finish	ENIG (Recommended)
Solder Mask	Matte Black
Silkscreen	White
Via Type	Through-hole
Minimum Track Width	As per JLCPCB standard

Parameter	Value
Minimum Clearance	As per JLCPCB standard

These specifications provide a balance between cost, durability, and manufacturing reliability.

12.5 PCB Fabrication

12.5.2 Surface Finish

The recommended surface finish for the Astra Flight Controller is ENIG (Electroless Nickel Immersion Gold). Advantages include:

- Excellent solderability.
- Flat component pads.
- Corrosion resistance.
- High reliability for fine-pitch components.
- Long storage life.

Although ENIG is slightly more expensive than HASL, it is better suited for components such as the STM32F411 and ICM-42688-P, which use fine-pitch packages.

12.6 Component Procurement

12.6.1 Ordering Components

Before assembly begins, all components listed in the BOM should be purchased. When ordering components:

- Verify manufacturer part numbers.
- Confirm package sizes.
- Check voltage ratings.
- Verify current ratings.
- Purchase from trusted suppliers.

Ordering incorrect package sizes is one of the most common causes of assembly delays.

12.6.2 Component Inspection

After components arrive, inspect them for:

- Physical damage.

- Correct package type.
- Correct value.
- Proper labeling.
- Moisture-sensitive packaging (if applicable).

Any damaged or incorrect components should be replaced before assembly begins.

12.7 Assembly Methods

12.7.1 Manual Assembly

For hobbyist builds and prototypes, manual assembly is often the most practical method. Required tools include:

- Fine-tip soldering iron.
- Hot air rework station.
- Flux.
- Fine tweezers.
- Solder paste.
- Magnification.
- Isopropyl alcohol for cleaning.

Manual assembly allows individual components to be placed and soldered carefully, making it ideal for small production runs and educational projects.

12.7.2 Automated Assembly

For larger quantities, automated assembly services can populate the PCB using the Pick-and-Place file and BOM. Advantages include:

- Higher placement accuracy.
- Faster production.
- Consistent solder quality.
- Reduced assembly time.

Services such as JLCPCB offer turnkey PCB assembly for compatible components, simplifying the manufacturing process for small batches.

12.8 Assembly Sequence

To reduce the risk of damaging components, assembly should follow a logical order:

1. Apply solder paste to all surface-mount pads.
2. Place the smallest passive components (resistors and capacitors).
3. Install integrated circuits.
4. Mount the inductor and power components.
5. Place connectors.
6. Reflow the PCB.
7. Inspect all solder joints.
8. Clean residual flux.
9. Perform electrical continuity checks.

Assembling components from the smallest to the largest improves access during soldering and reduces the chance of disturbing previously placed parts.

12.9 Soldering Inspection

After assembly, inspect the PCB carefully under magnification. Check for:

- Solder bridges.
- Cold solder joints.
- Tombstoned resistors or capacitors.
- Misaligned integrated circuits.
- Missing components.
- Excess solder.
- Insufficient solder.

Particular attention should be paid to the fine-pitch pins of the STM32F411 and the ICM-42688-P.